

TOMAC-PPO: 작업 단위로 학습하는 MEC 오프로딩

A Task Offloading and Resource Allocation Strategy Based on Multi-Agent Reinforcement Learning in Mobile Edge Computing

Guiwen Jiang, Rongxi Huang, Zhiming Bao, Gaocai Wang (2024)

동기화, 이동성 대응, 작업 지향 MDP와 PPO를 하나의 제어 구조로 연결한다

용어: 계산 위치와 학습 단위를 먼저 구분한다

용어	풀어 쓰기	이 발표에서의 역할
MEC	Mobile Edge Computing	단말 근처 서버 계산
MDP	Markov Decision Process	상태·행동·보상의 틀
MARL	Multi-Agent Reinforcement Learning	여러 대표 노드의 학습
Actor / Critic	정책망 / 가치망	행동 선택 / 가치 추정
TOMAC-PPO	Task-Oriented Multi-Agent Collaborative-PPO	작업 단위 공동 결정

PPO는 Proximal Policy Optimization; 정책을 갱신하는 방법이다

오프로딩과 자원 배분은 하나의 결정 문제다

실행 위치만 정해도 CPU, 통신, queue의 경쟁은 남는다

01 **작업 생성** 크기·기한·종류가 다름



02 **실행 위치** 단말·엣지·클라우드·폐기



03 **자원 배분** CPU·대역폭·대기 순서



04 **결과 평가** 지연·에너지·폐기 비용

실행 위치와 자원 비율을 따로 최적화하면 실제 queue와 통신 경쟁을 놓칠 수 있다

논문의 기여와 검증 범위

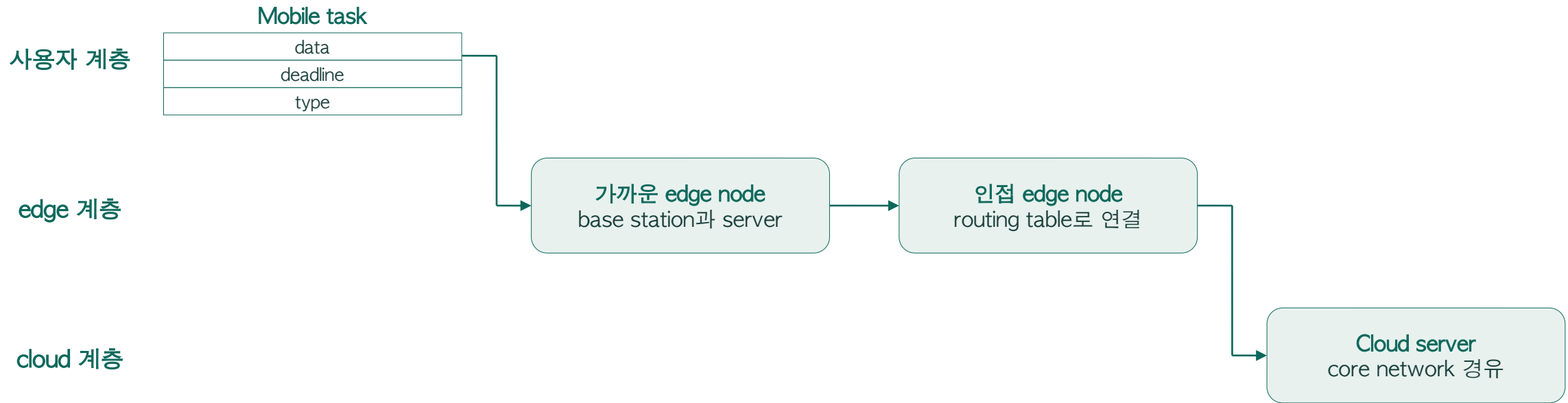
방법은 여러 구성 요소를 결합하지만 검증은 Python simulation에 한정된다

구성	논문이 제안한 내용	검증 경계
system	계층·대기열·장애 모델	실제 deployment가 아님
coordination	주기 동기화·이동 후 결과 반환	통신 overhead 측정 없음
decision	subnet별 Actor-Critic agent	global optimality 증명 없음
time	작업별로 겹치는 MDP step	credit assignment 보장 없음
learning	Transformer + PPO	Transformer ablation 없음

제안 구조와 시뮬레이션 결과를 실제 배포 성능 보장과 구분한다

경로 예: 이웃 엣지를 거쳐 클라우드로

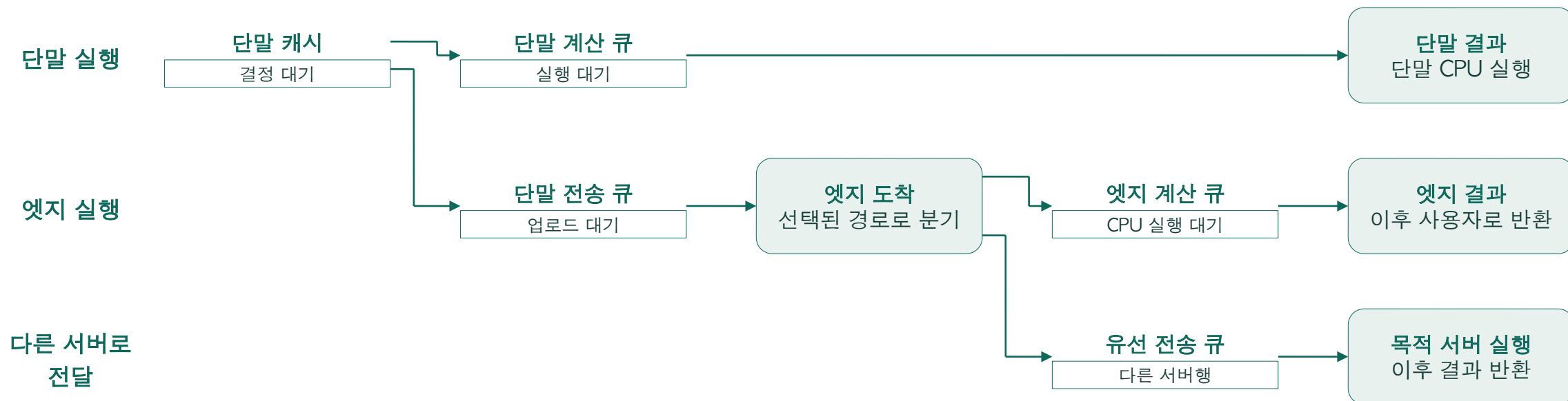
가능한 전달 경로 하나다. 가까운 엣지에서 끝나거나 코어망으로 바로 전달할 수도 있다



offload target은 한 edge에 한정되지 않으며 이웃 edge와 cloud도 후보가 된다

작업은 단말 실행·엣지 실행·전달 경로로 갈라진다

엣지에 도착한 작업을 실행할지 다른 서버로 전달할지 구분한다



다른 서버로 보낼 작업은 엣지 계산 큐를 먼저 거치지 않는다

네 task type은 비용에서 강조하는 항이 다르다

같은 policy가 모든 task에 같은 지연·에너지 trade-off를 적용하지 않는다

type	요구	phi1 지연	phi2 에너지	drop
high-priority	짧은 지연과 완료	사용	0	포함
critical	폐기 억제 요구	0	0	포함
low-priority	에너지 절감	0	사용	포함
routine	세 비용을 종합	1	1	포함

task type은 label에 그치지 않고 Eq. 11의 비용 가중치를 바꾼다

Action의 weight는 실제 자원 share로 정규화된다

agent가 직접 GHz나 MHz를 출력하지 않고 같은 server의 weight 합으로 나눈다

자원	action weight	실제 배분	함께 경쟁하는 대상
엣지 CPU	ff	$ff / \sum ff \times f_edge$	같은 엣지의 실행 작업
무선 대역폭	bf	$bf / \sum bf \times B$	같은 기지국의 사용자
큐 우선순위	qf	큰 값이 앞선다	같은 큐의 작업
큐 임계값	thmin, thmax	수용·확률 폐기·폐기	새 작업

개별 weight의 효과는 다른 사용자와 task가 낸 weight에 따라 달라진다

가중치 예시: 값 자체보다 경쟁자와의 비율이 중요하다

설명용 가정: CPU 총 처리량6, 대역폭20; 두 경쟁자의 가중치를 정규화한다

상황	가중치	정규화	배분
CPU A / B	.2 / .3	.2/.5, .3/.5	2.4 / 3.6
CPU B만 증가	.2 / .8	.2/ 1, .8/ 1	1.2 / 4.8
대역폭 U / V	.25 / .75	.25/1, .75/ 1	5 / 15 MHz
A의 관점	자기 .2는 그대로	B가 커지면 A 비율 감소	A 계산은 더 느려짐

A의 action이 그대로여도 B의 행동과 새 작업 도착이 A의 자원 몫을 바꾼다

큐 제어: 우선순위와 새 작업의 수용 여부는 다르다

가상 임계값 $thmin=4$, $thmax=8$ 로 원문의 세 구간을 읽는다

관측 또는 행동	예	처리	역할
낮은 혼잡	평균 큐 길이3	새 작업 수용	진입 허용
중간 혼잡	평균 큐 길이6	확률적으로 폐기	혼잡 완화
상한 도달	큐 길이8	새 작업 폐기	용량 보호
수용된 작업	$qf=.8$ vs $.3$	$.8$ 을 앞에 배치	실행 대기 순서

큰 qf 가 폐기 면제나 CPU 배분량을 직접 의미하지는 않는다

실행 경로마다 지연·단말 에너지 항이 다르다

전송, 전파, 계산, core forwarding을 경로에 맞게 더한다

항	원문 계산	의미
계산	$T = \lambda \rho / f$	입력량 × 계산 밀도/처리량
단말↔엣지	$T = \lambda / C + d/V_{space}$	무선 전송과 전파
엣지↔엣지	$T = \lambda / C_{fiber} + d/V_{fiber}$	인접 엣지 사이
엣지↔클라우드	$T = \text{전송} + \text{전파} + \text{core 지연}$	코어망 전달 포함
단말 에너지	$\kappa \lambda \rho f_{user}^2$ 또는 $P_{user} \times T$	단말 계산 또는 업로드

offload는 local CPU를 아끼지만 통신 지연과 전송 에너지를 새로 만든다

작업 비용: 지연·에너지·폐기를 합한다

Eq. 11의 각 항은 크기가 다른 측정값을 비교 가능한 형태로 묶는다

항	식	읽는 법	주의
delay	$\phi_1 \times T_{\text{total}}/\text{deadline}$	$T_{\text{total}} > \text{deadline}$ 이면 지각	type에 따라 제외
energy	$\phi_2 \times E_{\text{total}}/(\mu_1 \lambda^{\rho})$	0-1로 정규화	μ_1 값은 미제시
drop	$\text{drop} \in \{0,1\}$	discard이면 1	모든 type에 남음
total	$Z(x) = \text{delay} + \text{energy} + \text{drop}$	작을수록 유리	장기 합을 최소화

비용의 절대 크기보다 같은 normalization과 task type 아래의 비교가 중요하다

공동 최적화가 선택하는 변수와 지켜야 할 조건

논문은 긴 시간의 task cost 합을 줄이는 constrained problem을 세운다

범주	내용	대표 조건
offloading D	local·edge·cloud·drop 중 하나	각 task에 정확히 한 결과
resource ff, bf	CPU weight와 bandwidth weight	각 weight는 0부터 1
대기 순서 qf	작업 우선순위 가중치	단말·엣지 용량 제한
deadline	task total delay	T_{total} 은 deadline 이하
coverage	user와 edge 거리	base-station radius 이하

learned action의 목적은 Eq. 13 전체의 장기 비용을 낮추는 것이다

요구조건·감점·실제 충족은 같은 뜻이 아니다

총위

원문

읽어야 할 범위

문제의 요구

기한·용량·범위 제약

Eq13의 최적화 목표

학습의 감점

$PEN \in \{0, 1\}$ 을 보상에서 뺌

유한한 비용 신호

Critical 요구

폐기하면 안 됨

작업 유형의 설계 요구

실험 결과

Critical drop = 5.50%

무폐기를 달성한 것은 아님

제약 위반에 감점을 주었다는 사실만으로 모든 행동의 제약 준수를 증명할 수 없다

주기 동기화가 local observation을 보완한다

단말·엣지·클라우드의 주기 보고와 미수신 timeout 규칙을 묶어 본다

통신 대상	주기적으로 공유하는 정보	해석
단말 → 엣지	단말의 현재 위치	이동 상태 갱신
엣지 ↔ 인접 엣지	상태·연결 정보	경로 정보 유지
클라우드 ↔ 엣지	연결 상태	연결 확인
미수신 처리	동기화 주기를 넘는 timeout	연결 끊김 판정·재전송 없음

policy state의 network view는 실시간 전역 관측이 아니라 주기적으로 동기화된 정보다

이동 사용자 scheduling의 앞부분

task summary로 먼저 결정하고 실제 data는 결정 뒤에 보낸다

01 1. summary 보고 user가 가까운 decision node에 task 정보를 전송



02 2. decision 반환 node가 network 상태를 보고 offload 결과 계산



03 3. 실행 시작 user가 local 계산 또는 task data 전송

초기 signaling과 실제 task-data 전송은 서로 다른 단계다

이동 사용자 scheduling의 반환 경로

실행 중 user가 cell을 바꾸면 결과는 현재 위치에 가까운 node로 돌아온다

01 4. target 전달 decision node가 선택된 실행 node로 task 전송



02 5. return 선택 실행 node가 현재 user에 가까운 node를 찾음

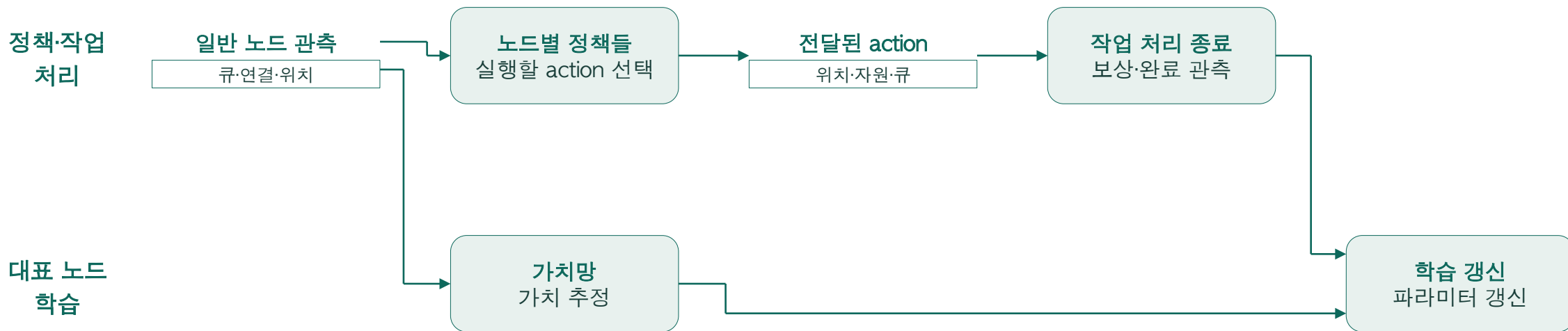


03 6. 결과 전달 shortest path 뒤 return node가 user에게 전송

결정을 내린 node, 실행 node, 결과를 돌려주는 node가 서로 다를 수 있다

Subnet마다 commentator node 하나가 agent가 된다

ordinary node의 관측과 실행을 subnet 대표 edge가 모아 결정한다



전체 network가 한 agent인 구조도, 모든 edge가 독립 agent인 구조도 아니다

분산 실행과 지역 중앙집중의 경계

지역별 대표 노드의 결정 구조이며 전역 중앙 학습기나 파라미터 평균을 가정하지 않는다

질문	원문이 말하는 것	원문이 말하지 않는 것
agent 단위	subnet의 commentator node	엣지마다 독립 agent 아님
ordinary node	관측 제출 후 action 실행	자기 policy로 결정하지 않음
critic	commentator 내부 value network	전역 critic은 미제시
병렬성	Algorithm 1의 agent loop	파라미터 평균은 미제시
cloud 역할	계산 후보와 connectivity 확인	전체 정책 학습기 아님

TOMAC-PPO의 분산성은 subnet 간 결정 분리에 있다

고정 global slot 대신 task마다 다른 step을 둔다

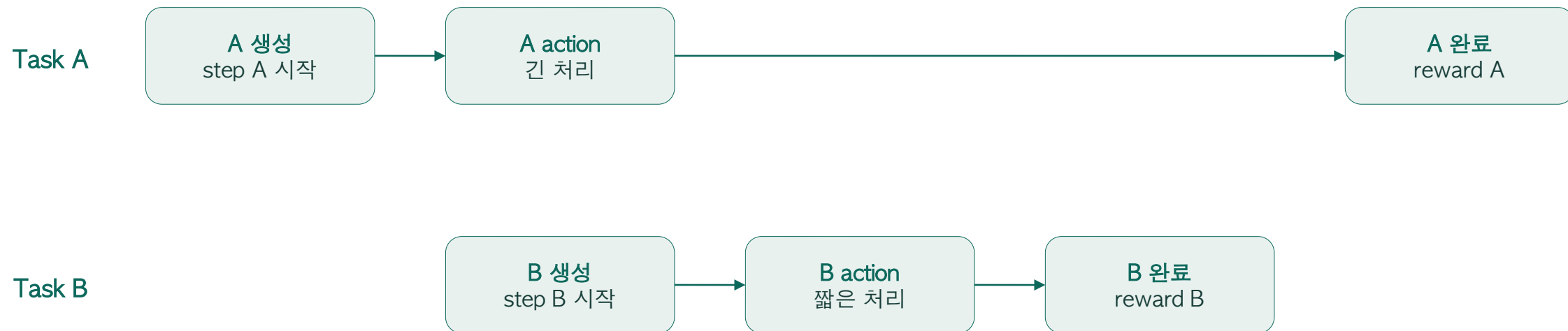
처리 시간이 다른 task를 같은 clock tick에 맞추지 않는다

구분	고정 시간 slot	TOMDP task step
시작	모든 system이 같은 경계	각 task 생성 시점
종료	정해진 slot 끝	해당 task 완료 시점
길이	일정	task 처리 시간에 따라 다름
동시성	한 global tick으로 표현	여러 task step이 겹침
reward	slot 단위로 설계	task 완료 뒤 feedback

TOMDP의 핵심은 task lifecycle과 transition의 경계를 맞추는 것이다

Task step은 서로 겹치고 완료 순서도 바뀔 수 있다

설명용 사건 순서: A 생성 → B 생성 → B 완료 → A 완료; 축은 실제 시간 단위가 아니다



각 작업의 생성·관측·완료 시점을 서로 구분한다

TOMDP state는 task와 연결 가능한 network를 함께 본다

Eq. 14는 tensor shape보다 정보 범주를 읽는 것이 안전하다

범주	state 항	의미
task	정규화 λ , deadline, task type	처리량과 중요도
mobility	user coordinate records H_i	edge가 저장한 위치
user queue	단말의 세 큐 합 / R_{ud}	단말의 혼잡
edge queue	엣지 두 큐 합 / R_{en}	선택 edge의 혼잡
network	$Link_j$, $thmin$, $thmax$, j/n	연결과 queue control

한 agent의 state는 local observation과 동기화로 얻은 연결 node 정보의 결합이다

TOMDP action은 위치와 네 종류의 control을 묶는다

Eq. 15의 한 action이 실행 경로와 자원 경쟁을 동시에 바꾼다

action 항	결정	system에 미치는 영향
D	local·edge·cloud·drop	실행 경로
ff	CPU allocation weight	edge compute delay
qf	queue priority weight	대기 순서
bf	bandwidth weight	uplink rate
thmin, thmax	queue thresholds	accept와 drop policy

행동은 실행 위치뿐 아니라 CPU·대역폭·큐 제어를 함께 포함한다

Reward는 현재 task cost를 user 평균과 비교한다

Z_{avg} 는 사용자 평균 작업 비용이다. Z/Z_{avg} 가 작을수록 보상이 커지고 $PEN=1$ 이면 1을 뺀다

항	값	역할
기본 상수	$1 + 1/2$	reward의 기준점을 1.5로 둔다
상대 cost	Z / Z_{avg}	현재 task가 평균보다 비싼지 비교
constraint	$PEN(x) \in \{0, 1\}$	initial decision 위반을 감점
전체	$R = 1.5 - Z/Z_{avg} - PEN$	task 완료 뒤 계산

reward는 절대 latency가 아니라 task type이 반영된 정규화 cost를 사용한다

구성 예시: 한 task의 cost와 reward 계산

가상 예: 대기·전파·결과 반환 지연은 0으로 두고 단말 에너지 정규화 항은 0.10으로 정한다

단계	입력	계산	결과
계산	$2 \text{ Mbit} \times 0.3 \text{ Gcyc/Mbit}$	$0.6 \text{ Gcyc} / 1.2 \text{ GHz}$	0.50 s
uplink	rate=10 Mbit/s	$2 / 10$	0.20 s
routine cost	deadline=1 s, energy term=0.10	$0.70 / 1 + 0.10 + 0$	$Z=0.80$
reward	average cost=1.00, PEN=0	$1.5 - 0.80 / 1.00$	$R=0.70$

같은 task도 배정 CPU와 uplink rate가 바뀌면 cost와 reward가 함께 바뀐다

한 task가 하나의 learning transition이 되는 순서

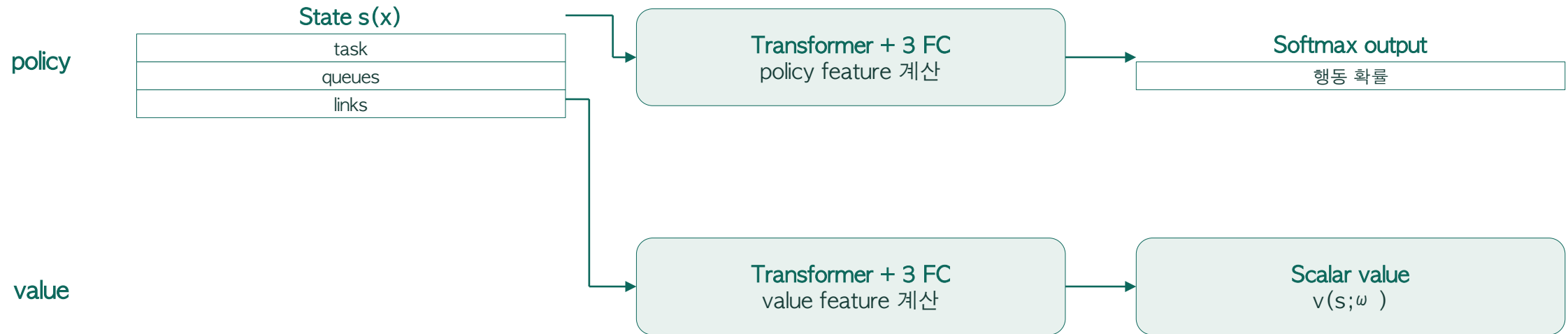
action과 reward 사이의 실제 처리 시간을 task step 안에 포함한다

01	관측 시점	요약·네트워크 상태 관측
↓		
02	정책 결정	실행 위치·가중치 선택
↓		
03	동시 처리	다른 작업과 자원 공유
↓		
04	처리 종료	해당 작업의 결과 확정
↓		
05	학습 자료	보상·완료 상태 연결

reward delay를 없애는 것이 아니라 어느 task action에 연결할지 명시한다

같은 state가 policy와 value의 두 network로 들어간다

policy는 action probability를, value network는 state value 하나를 출력한다



policy와 value는 출력 목적이 다르며 target value network는 value 구조를 복제한다

Transformer: 구조의 위치와 미제시 세부

논문이 명시한 구조와 구현에 필요한 미제시 항목을 분리한다

구분	명시됨	미제시
목적	state memory와 prediction을 지원	prediction target과 평가 지표
위치	state 뒤, FC layers 앞	encoder-decoder 세부
attention	Transformer 사용	head·layer·hidden size
sequence	state s가 입력	history 길이와 token 구성
효과	전체 TOMAC-PPO 결과	Transformer 제거 ablation

Transformer의 독립 효과나 장기 예측 정확도는 이 실험으로 확인할 수 없다

PPO는 확률을 강제 고정하지 않고 학습 목적식을 자른다

엔트로피를 잠시 제외하면 $J = \min(\text{ratio} \times \text{Adv}, \text{clip}(\text{ratio}, 0.8, 1.2) \times \text{Adv})$

기호	정의	역할
ratio_t	$\pi_{\text{new}}(a_t s_t) / \pi_{\text{old}}(a_t s_t)$	action probability 변화
zeta	clip 반경, Table 1은 0.2	목적식 안의 clip 구간
Adv_t	$Q\pi(s_t, a_t) - V\pi(s_t)$	action이 기대보다 좋은 정도
clip term	$\text{clip}(\text{ratio}, 1-\text{zeta}, 1+\text{zeta}) \times \text{Adv}$	surrogate 증가 제한
entropy	$cH(s_t; \theta), c=0.01$	exploration을 장려

실제 ratio가 구간 밖으로 갈 수 있다; clip은 확률 변화의 강제 상한이 아니다

구성 예시: return, advantage, clipped surrogate

entropy를 제외한 산술 예시이며 논문의 training log가 아니다

단계	계산	결과	해석
discount return	$0.4 + .95 \times .2 + .95^2 \times 1$	$u=1.4925$	세 reward를 합침
advantage	$u - v = 1.4925 - 1.10$	$Adv'=0.3925$	positive action
policy ratio	$0.56 / 0.40$	$ratio=1.40$	probability 40% 증가
clip	$clip(1.40, 0.8, 1.2)$	1.20	목적식의 clip 값
surrogate	$min(1.40, 1.20) \times 0.3925$	0.4710	raw 0.5495보다 작음

positive advantage라도 ratio가 clip 범위를 넘으면 objective가 더 커지지 않는다

음의 advantage에서는 반대쪽 구간이 평평해진다

구성 예: $Adv = -1$, clip 구간 $[0.8, 1.2]$, 엔트로피 제외; 큰 J가 유리하다

ratio	raw = ratio $\times$ Adv	clipped $\times$ Adv	J = 두 값의 min
0.6	-0.6	-0.8	-0.8
0.8	-0.8	-0.8	-0.8
1.0	-1.0	-1.0	-1.0
1.4	-1.4	-1.2	-1.4

나쁜 행동의 확률을 더 낮춰도 $ratio < 0.8$ 이면 이 항에서 추가 이득이 없다

Algorithm 1의 training loop

각 subnet agent가 trajectory를 모으고 policy와 value를 갱신한다

01	초기화	정책·가치·target 가치망
↓		
02	Episode	학습 회차 반복
↓		
03	Agent별 수집	각 정책으로 T-step 수집
↓		
04	정책 갱신	Adv 계산 후 목적식 상승
↓		
05	가치망 갱신	가치·target 가중치 갱신

Algorithm 1에는 global parameter server나 agent 간 gradient averaging이 없다

원문 수식의 재현 경계: 부호·엔트로피·target

원문 표기의 모호함과 빠진 계산 비용을 그대로 경계로 남긴다

항목	원문	교육적 해석
value loss	$L = -\sum (u-v)^2$	최소화하면 오차가 커질 수 있음
entropy 위치	min의 두 번째 항에 +cH	min 밖 +cH와 다름
target 갱신	$\hat{w} \leftarrow \sigma w + (1-\sigma) \hat{w}$	출력 사용처는 미제시
복잡도	$O(n \text{ emax})$	trajectory·망 연산 비용 생략
병렬성	agent들을 병렬 처리	실행 시간이 n배라는 뜻 아님

Eq. 21의 부호는 임의로 고치지 말고 구현 전 원저자 확인이 필요하다

구현을 재현하려면 추가로 필요한 연결 규칙

경계

출력→행동

명시된 요소

Softmax / 이산 D·연속 가중치

빠진 연결

인코딩·샘플링·제약 수정

동시 작업→trajectory

겹친 step / T-step return

순서·다음 상태 연결

보상 분모

생성 작업의 사용자 평균 비용

미완료·초기·0평균 처리

입력 생성

Table1: N(5100) Mbit

분포의 정확한 매개변수

논문에 없는 연결 규칙을 임의로 채우면 원문 재현이 아니라 별도 구현이 된다

비교 baseline은 학습 구조와 실행 위치가 다르다

Figure legend의 A3C는 본문에서 정의한 TO-A3C baseline을 가리킨다

방법	결정 구조	특징	비교에서의 역할
TOMAC-A2C	task-oriented multi-agent	A2C update	가까운 학습 기준
TO-A3C	독립 single-agent 환경 병렬	multi-agent system 없음	parallel RL 비교
CCP	cloud 우선	가능하면 상위 계층 처리	offload heuristic
LC	local only	information report 생략	단말 실행 기준

TOMAC-A2C가 가장 가까운 학습 baseline이고 CCP와 LC는 위치 heuristic이다

실험 결과가 성립한 simulation 조건

현장 network가 아니라 저자가 구성한 Python environment의 결과다

범주	원문 설정	해석 경계
네트워크	엣지 10개·반경 100m·20MHz	시뮬레이션
계산	엣지 2.4GHz·14코어 / cloud 10GHz	실물 성능 측정 아님
작업 비율	high=.2, critical=.2, low=.2, routine=.4	고정 생성 확률
학습	1000회· $\gamma = .95$ ·학습률 3×10^{-4}	반복 seed 수 미제시
갱신	clip=.2·entropy=.01·target=.01	세부 구조 미제시

수치 비교는 같은 simulator와 task mix 안에서만 직접 해석할 수 있다

Figure 10: PPO는 더 높은 reward에 이르지만 더 흔들린다

50 users와 failure probability 0.1 %에서 TOMAC-A2C와 비교한다

관찰	TOMAC-A2C	TOMAC-PPO	읽는 경계
moving average	episode와 함께 증가	episode와 함께 증가	training curve
final reward	낮은 plateau	약 23.9% 높다고 보고	본문의 조건부 비교
fluctuation	상대적으로 작음	더 큼	entropy exploration 가설
evidence	한 plotted curve	한 plotted curve	seed와 error bar 없음

더 높은 plotted reward와 더 안정적인 학습은 같은 주장이 아니다

Figure 11 : user 수가 늘어도 TOMAC-PPO cost가 가장 낮다

failure probability를 0.1%로 고정하고 users 5명부터 50명까지 바꾼다

조건	관찰	50 users 결과	한계
network load	user 5-50	TOMAC-PPO가 최저	한 topology model
comparison	다섯 curve	baseline보다 19.4%-66.6% 낮음	원문 보고 범위
nearest baseline	TOMAC-A2C	gap은 상대적으로 작음	PPO 외 구성 공유
evidence	average task cost	single curve values	분산과 CI 없음

19.4%-66.6%는 50-user simulator에서 baseline별로 달라지는 감소 범위다

장애 확률이 극단적이면 학습 방식의 이점이 줄어든다

50 users에서 edge-line failure probability를 10%부터 100%까지 바꾼다

failure 구간	TOMAC 계열	다른 방법	해석
10%–60%	낮은 average cost	CCP·A3C·LC는 더 높은 비용	edge path 사용 가능
70% 부근	cost가 급격히 상승	curve 간 gap 감소	available edge 감소
100%	약 2.3	LC 2.24	edge layer disconnected
CCP < 50%	해당 없음	일시적으로 cost 감소	엣지 실행 증가
evidence	plotted averages	plotted averages	반복·CI 없음

모든 edge가 끊기면 offloading policy가 선택할 수 있는 유효 경로도 사라진다

Figure 13: task type마다 비교 metric이 다르다

50 users와 failure probability 0.1%에서 다섯 방법의 값을 읽는다

방법	routine cost	high delay(s)	low energy(J)	critical drop
LC	2.23	26.97	3.54e-6	36.08%
CCP	2.80	31.98	0.62	62.64%
A3C	1.78	24.92	0.44	24.33%
TOMAC-A2C	1.16	13.47	0.27	9.30%
TOMAC-PPO	0.94	10.86	0.25	5.50%

TOMAC-PPO는 세 metric에서 최저지만 low-priority energy는 LC가 가장 낮다

결과를 일반화하기 전에 남는 한계

논문이 밝힌 한계와 실험 보고에서 확인되지 않은 항목을 함께 구분한다

범주	확인된 한계	결론에 미치는 영향
simulation	Python 기반 단순 MEC model	현장 latency와 failure를 보장하지 않음
scale	agent 수 증가와 정보 lag	large network scalability 미확인
reproducibility	dataset identity·seed·code 미제시	variance와 재현성 평가 제한
architecture	Transformer 세부와 ablation 없음	성능 원인을 분리할 수 없음
theory	Eq. 21 부호 모호, formal theorem 없음	구현과 optimality 보장 아님

TOMAC-PPO는 promising simulation design이지 검증 완료된 production controller가 아니다

TOMAC-PPO를 한 흐름으로 다시 읽기

network coordination과 task lifecycle을 먼저 정의한 뒤 PPO가 joint action을 학습한다

01	상태 공유	위치·연결을 주기 보고
↓		
02	지역 대표	subnet별로 대신 결정
↓		
03	작업 단위	생성·관측·완료 구분
↓		
04	공동 행동	위치·자원·큐 제어
↓		
05	학습	종료 보상으로 갱신

이 논문의 핵심은 PPO 하나보다 task, network, time의 경계를 함께 설계한 데 있다